

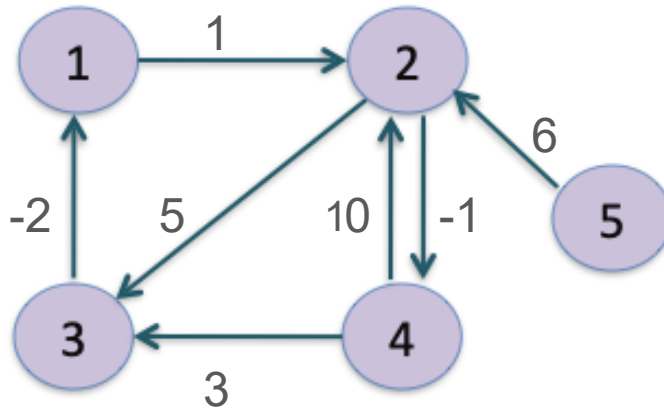
Dynamic Programming for Shortest Paths in Graphs

Did somebody
say G-raph?

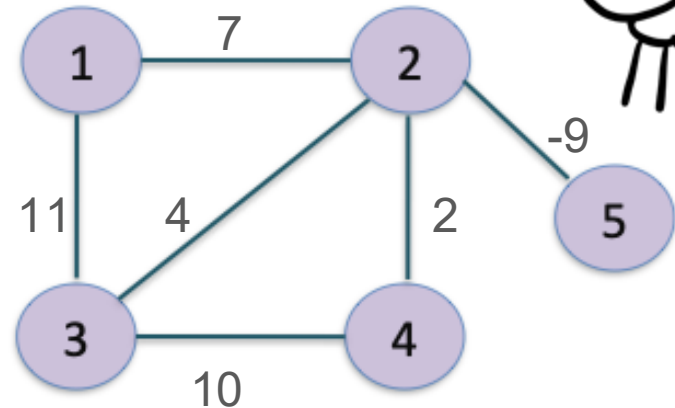


Directed and undirected graphs

Directed graph



Undirected graph



Why do we even care about negative weights?



Distance from s to t (denoted $\text{dist}(s,t)$) = minimum over all paths P from s to t , of the sum of the edge weights along P .

$\text{dist}(s,t) = \infty$ if there's no path from s to t .

Representation: Adjacency list

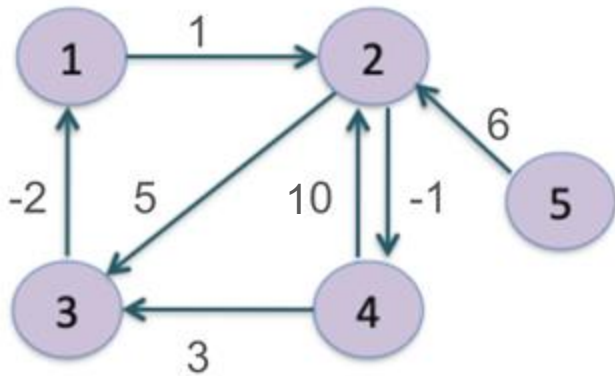
Notation: V = vertex set, E = edge set, $n = |V|$, $m = |E|$

The shortest paths problems we'll consider

Input: Directed weighted graph. Weights can be negative but assume no negative-weight cycles.

Single-Source Shortest Paths (SSSP): Given a “source” vertex s , find a shortest path from s to every vertex t .

All-Pairs Shortest Paths (APSP): For every pair s, t of vertices, find a shortest path from s to t .



	1	2	3	4	5
1	0	1	3	0	∞
2	0	0	2	-1	∞
3	-2	-1	0	-2	∞
4	1	2	3	0	∞
5	6	6	8	5	0

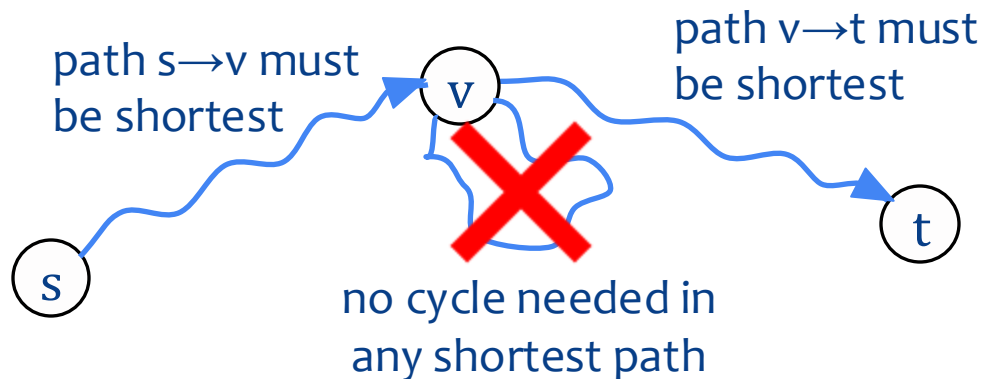
What about
single-pair
shortest path?



Two Key Observations

Principle of
Optimal
Substructure

1. If a shortest path from s to t goes through vertex v , then it must take a **shortest path from s to v** , then a **shortest path from v to t** .
2. Since there are no negative-weight cycles in the graph, there is a shortest path from s to t with **no cycles** in it.



Check out my recurrence for SSSP!

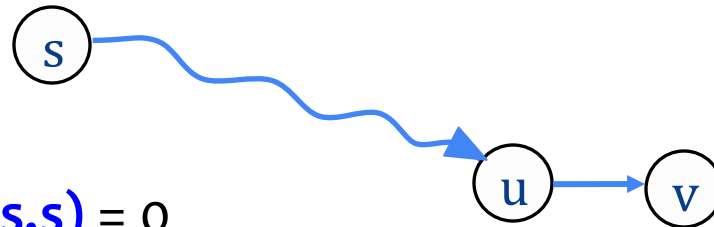


Your equation is technically correct, but it's not really a recurrence and it doesn't work for DP.

$$\text{dist}(s,v) = \min_{(u,v) \in E} \{ \text{dist}(s,u) + w(u,v) \}$$

*dist(s,v) uses dist(s,u)
dist(s,u) uses dist(s,v)
circular reasoning!*

In a shortest $s \rightarrow v$ path, u is the last vertex before v



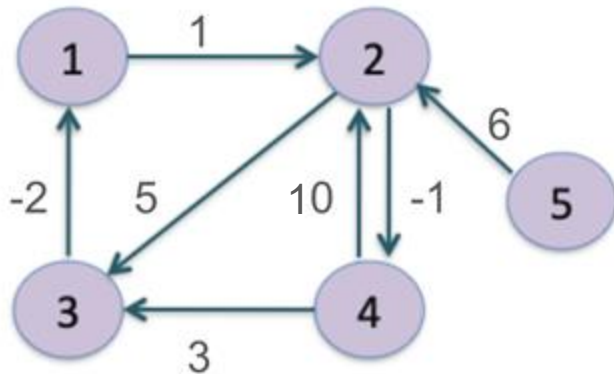
Base case: $\text{dist}(s,s) = 0$

Notation:

- $w(y,z)$ is the weight of the edge $y \rightarrow z$
- $\text{dist}(y,z)$ is the distance from y to z

Recurrence for SSSP

Key Definition. Let $\text{dist}^{(i)}(s,v)$ be the length of a shortest s-to-v path that uses at most i edges, or ∞ if there's no such path.



What is...

$\text{dist}^{(0)}(5,3)?$ ∞

$\text{dist}^{(1)}(5,3)?$ ∞

$\text{dist}^{(2)}(5,3)?$ 11

$\text{dist}^{(3)}(5,3)?$ 8

$\text{dist}^{(4)}(5,3)?$ 8

Recurrence for SSSP

Key Definition. Let $\text{dist}^{(i)}(s,v)$ be the length of a shortest s-to-v path that uses at most i edges, or ∞ if there's no such path.

End goal: $\text{dist}^{(n-1)}(s,v)$ for each v . Why?

Recurrence:

$$\text{dist}^{(i)}(s,v) = \min \left\{ \text{dist}^{(i-1)}(s,v), \min_{(u,v) \in E} \{ \text{dist}^{(i-1)}(s,u) + w(u,v) \} \right\}$$

for e.g. $\text{dist}^{[1]}(s,s)$

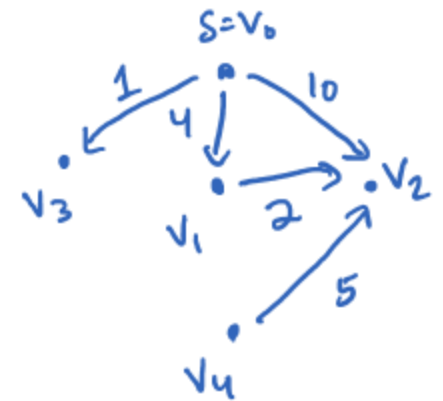
Base case(s): $\text{dist}^{(0)}(s,v) = \begin{cases} \infty & \text{if } s \neq v \\ 0 & \text{if } s = v \end{cases}$



The recurrence takes
inspiration from Professor
Y's first attempt

Let's follow the DP Recipe

	$i=0$	1	2	3 ...	$n-1$
$S=v_0$	0	0			
v_1	∞	4			
v_2	∞	10	6		
v_3	∞	1			
v_4	∞	∞			
$\vdots$					



$$\min \{10, 0+10, 4+2, \infty+5\} = 6$$

n^2
 (# cells)
 • (time per cell)
 $\Rightarrow O(n^3)$

Running time:

(# columns) • (time per column)
 $\Rightarrow O(nm)$

base cases

final answers

This is called the **Bellman-Ford Algorithm**
 (Bellman, Ford, Moore, Shimbel, 1955)

The DP Recipe

1. Define *value version* of what we want to compute
2. Derive a recurrence
3. Size of table: How many dimensions? Range of each dimension?
4. What are the base case(s)?
5. To fill in a cell, which other cells need to be filled already? In which order do I fill the table?
6. Which cell(s) contain the final answer?
7. Running time is often (size of table) $\cdot$ (time to fill each entry)
8. To reconstruct the solution (instead of just its value) follow arrows from final answer to base case

Pseudocode for Bellman-Ford

Algorithm SSSP(G, s)

table := 2D-array indexed from 0 to $n-1$ in both dimensions

// first dimension represents vertices $v_0, \dots, v_{n-1}$, where $s=v_0$,
second dimension represents the i in $\text{dist}^{(i)}$

table(0,0) = 0 // base case for s

for $v = 1$ to $n-1$: **table**($v, 0$) = ∞ // rest of base cases

for $i = 1$ to $n-1$: // for each i (corresponding to $\text{dist}^{(i)}$)

for $v = 0$ to $n-1$: // for each vertex

table(v, i) = $\min \left\{ \min_{(u,v) \in E} \{ \mathbf{table}(u, i-1) + w(u,v) \}, \mathbf{table}(v, i-1) \right\}$

Return, for each $v \in V$, **table**($v, n-1$) as $\text{dist}(s,v)$

Structure of **all** DP pseudocode: loop through table in appropriate order and fill in cells according to recurrence.

Previously we assumed the graph has no negative-weight cycles. **But what if it does?**

With a slight modification, the Bellman-Ford (BF) Algorithm can **detect** whether or not there is a negative-weight cycle:

Run BF for n more iterations!

I.e calculate $\text{dist}^{(i)}(s,v)$ for all i up to $2n-1$.

Claim: There is a negative-weight cycle (reachable from s) if and only if $\text{dist}^{(2n-1)}(s,v) < \text{dist}^{(n-1)}(s,v)$ for some vertex v .

*Actually it suffices to run BF for only 1 more iteration (n iterations total).

Faster Algorithms for SSSP

My postdoc
advisor ↘

- Bernstein, Nanongkai, Wulff-Nilsen ('22): $O(m \cdot \log^8 n)$ ← integer weights
- Fineman ('24), ..., Hair, Li, Li, Zhang ('26): $O(m^{1+o(1)})$ ← any weights
- If no negative weights:
 - Dijkstra's algorithm ('59): $O(m + n \log n)$
 - Duan, Mao, Mao, Shu, Yin ('26): $O(m \cdot \log^{1/2} n + \sqrt{mn} \cdot \log n \cdot \log \log n)$

UM alum ↗

Initial idea for solving APSP: Run SSSP from every vertex.

That works, but the algorithm you're about to see is faster for dense graphs: $O(n^3)$ BF from every source: $O(mn^2)$

Let's try essentially the same recurrence as BF

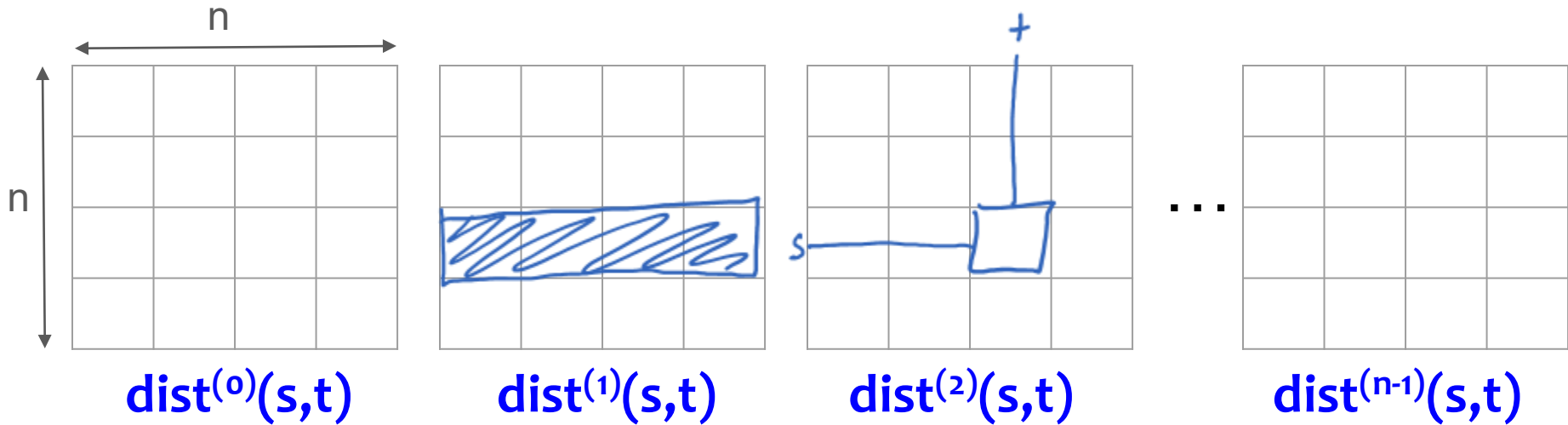
$$\text{dist}^{(i)}(s,v) = \min \{ \min_{(u,v) \in E} \{ \text{dist}^{(i-1)}(s,u) + w(u,v) \}, \text{dist}^{(i-1)}(s,u) \}$$

$$\text{Base case: } \text{dist}^{(0)}(s,v) = \begin{cases} 0 & \text{if } s = v \\ \infty & \text{otherwise} \end{cases}$$

The only difference:

there are 3 “free” variables: s , v , and $i \Rightarrow 3D$ DP table!

The DP table



Running time: $O(n^4)$, $O(mn^2)$, same as BF from every source

But there's a cool trick that allows each cell to only look at 3 other cells...

Old idea: i denotes *length* of permissible paths.

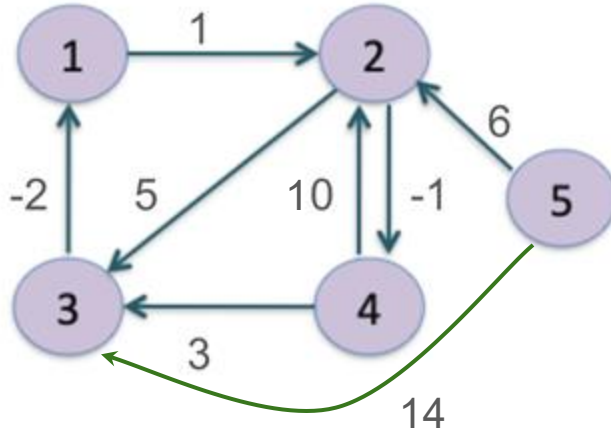
New idea: new i denotes *allowable intermediate vertices*.

Arbitrary ordering of vertices: $v_1, v_2, \dots, v_n$.

Key Definition: Let $\text{dist}^{[i]}(s,t)$ be the length of a shortest s -to- t path that's allowed to use vertices $\{s,t\} \cup \{v_1, \dots, v_i\}$ (not necessarily all of them, and in any order)



These are the allowed **layovers**!
(we don't have to use them all)



What is... $\text{dist}^{[0]}(5,3)?$ 14

$\text{dist}^{[1]}(5,3)?$ 14

$\text{dist}^{[2]}(5,3)?$ 11

$\text{dist}^{[3]}(5,3)?$ 11

$\text{dist}^{[4]}(5,3)?$ 8

Recurrence

Arbitrary ordering of vertices: $v_1, v_2, \dots, v_n$.

Key Definition: Let $\text{dist}^{[i]}(s,t)$ be the length of a shortest s -to- t path that's allowed to use vertices $\{s,t\} \cup \{v_1, \dots, v_i\}$ (not necessarily all of them, and in any order)

Base case: $\text{dist}^{[0]}(s,t) = \begin{cases} 0 & \text{if } s = t \\ w(s,t) & \text{if } (s,t) \in E \\ \infty & \text{otherwise} \end{cases}$

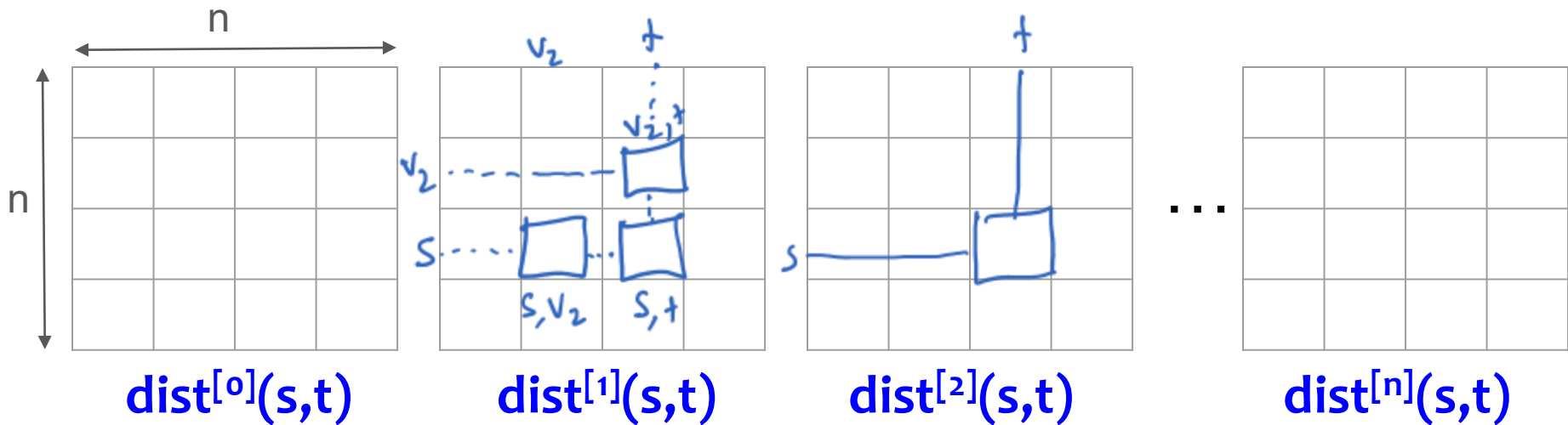


$$\text{dist}^{[i]}(s,t) = \min \left\{ \underbrace{\text{dist}^{[i-1]}(s,t)}_{\text{Case 1 ("Lose it!")}}, \underbrace{\text{dist}^{[i-1]}(s,v_i) + \text{dist}^{[i-1]}(v_i,t)}_{\text{Case 2 ("Use it!")}} \right\}$$

Case 1 (“**Lose it!**”):
shortest st -path
doesn't contain v_i

Case 2 (“**Use it!**”):
shortest st -path
contains v_i

Let's Follow the DP Recipe



Running time: $\underbrace{(\# \text{cells})}_{n^3} \cdot \underbrace{(\text{time per cell})}_{O(1)} = O(n^3)$

This is called the **Floyd-Warshall Algorithm** (1962)

Pseudocode for Floyd–Warshall

Algorithm APSP(G)

table := 3D-array ($1..n, 1..n, 0..n$)

// first two dimensions represent vertices $1, \dots, n$,

third dimension represents restricting to the first i internal vertices

for $s = 1$ to n :

 for $t = 1$ to n :

$\text{table}(s, t, 0) = 0$ if $s=t$, $w(s, t)$ if $(s, t) \in E$, and ∞ otherwise

for $i = 1$ to n :

 for $s = 1$ to n :

 for $t = 1$ to n :

$\text{table}(s, t, i) = \min\{\text{table}(s, t, i-1), \text{table}(s, i, i-1) + \text{table}(i, t, i-1)\}$

Return $\text{table}(s, t, n)$ for all s, t

A question to ponder...

Why doesn't the trick of computing $\text{dist}^{[i]}(s,t)$ give a faster-than-BF algorithm for SSSP too?

Progress on APSP since Floyd-Warshall

Author	Runtime	Year
Fredman	$n^3 \log \log^{1/3} n / \log^{1/3} n$	1976
Takaoka	$n^3 \log \log^{1/2} n / \log^{1/2} n$	1992
Dobosiewicz	$n^3 / \log^{1/2} n$	1992
Han	$n^3 \log \log^{5/7} n / \log^{5/7} n$	2004
Takaoka	$n^3 \log \log^2 n / \log n$	2004
Zwick	$n^3 \log \log^{1/2} n / \log n$	2004
Chan	$n^3 / \log n$	2005
Han	$n^3 \log \log^{5/4} n / \log^{5/4} n$	2006
Chan	$n^3 \log \log^3 n / \log^2 n$	2007
Han, Takaoka	$n^3 \log \log n / \log^2 n$	2012
Williams	$n^3 / \exp(\sqrt{\log n})$	2014



This is
wild!



Conclusion: Maybe $O(n^{2.99})$ is impossible?

Maybe $O(n^{2.99})$ is impossible?

Either **ALL** of the following have $O(n^{<3})$ time algorithms or **NONE** of them do: (Virginia Vassilevska Williams, Ryan Williams, 2010)

1. APSP
2. Minimum Weight Triangle
3. Metricity
4. Minimum Cycle
5. Distance Product
6. Second Shortest Path
7. Replacement Paths
8. Negative Triangle Listing

...

